Exercise 1B: Custom Request Logging Middleware

Context: Order is fixed, but you still cannot tie log lines to a single failing request. Implement middleware that logs start/finish and stamps X-Correlation-Id.

Your Task (implement, do not transcribe a full solution)

1. Add `RequestLoggingMiddleware.cs` with a `RequestDelegate next` and `ILogger<RequestLoggingMiddleware>`.
2. In `InvokeAsync`:
    - Generate a short correlation id (for example from `Guid.NewGuid().ToString("N")[..8]`).
    - Set `context.Response.Headers["X-Correlation-Id"]` before `await next(context)`.
    - Use `Stopwatch` to measure elapsed time.
    - Log one line on entry (method, path, correlation id) and one on exit (status code, elapsed ms, same id).
3. In `Program.cs`, register in this order (adjust only if your facilitator's template differs):
    - `app.UseMiddleware<RequestLoggingMiddleware>();` first (outer wrapper).
    - Then `UseExceptionHandler`, `UseHttpsRedirection`, `UseRouting`, `UseAuthentication`, `UseAuthorization`.
    - Map `GET /api/assessments/results` last, still with `.RequireAuthorization()`.

If you are unsure where UseExceptionHandler fits, use Module 4 ASP.NET Core Fundamentals (middleware and error-handling sections) or your Session 3 materials—you are wiring the slot now so later exercises can plug in ProblemDetails without reordering everything.

Run / verify / common failure

- **Run / verify / common failure**
    - **Run:** `dotnet run`
    - **Check:** Same anonymous GET as Exercise 1 (`/api/assessments/results`).
    - **Expected:** Response still 401; response headers include `X-Correlation-Id`; two log lines per request sharing the same correlation id.
    - **Common failure no logs:** Middleware registered after the endpoint.
    - **Common failure no header:** Header set after `await next(context)` (too late).